Ders 10 Genişletilmiş Notu: Sınav Sorularının Çözümü, İkili Arama Ağaçları, Radix ve Kırmızı-Siyah Ağaçlar, B-Ağaçlarına Giriş

Ders 10 Genişletilmiş Notu: Sınav Sorularının Çözümü, İkili Arama Ağaçları, Radix ve Kırmızı-Siyah Ağaçlar, B-Ağaçlarına Giriş

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Sınav Sorularının Çözümü
- 2. İkili Arama Ağaçlarına Giriş
- 3. Arama, Minimum ve Maksimum
- 4. Ardıl ve Öncül
- 5. Ekleme
- 6. Silme ve TRANSPLANT
- 7. Tekrar Eden Anahtarlar
- 8. Radix Ağaçları
- 9. Kırmızı-Siyah Ağaçlar
- 10. Kırmızı-Siyah Ağacın Yüksekliği
- 11. Döndürme ve Kırmızı-Siyah Ağaca Ekleme
- 12. B-Ağaçlarına Giriş
- Örnek Sorular

1. Sınav Sorularının Çözümü

Hoca derse, sınav kâğıtlarına hızlıca göz attığını ve soruların beklenenden zor bulunduğunu düşündüğünü söyleyerek başladı; bu yüzden soruları Microsoft Whiteboard üzerinde kısaca çözdü. Soruların tam metni ve çözümleri "Örnek Sorular" bölümündedir. Burada her sorunun hangi kavramı sınadığı anlatılıyor.

1.1 Ana teoremi tersten uygulamak

İlk soru, çalışma zamanı $n^{\log_b a}$ olan bir yineleme bağıntısı (recurrence relation) kurmayı, bunun ağacını çizmeyi ve her seviyedeki işlem sayısıyla toplam işlem sayısını bulmayı istiyordu. Çoğu öğrenci bağıntının genel biçimini doğru yazmıştı (ekrandan, 00:06:40):

$$T(n) = aT(n/b) + O(n^d)$$

Burada $O(n^d)$ terimi, yani $f(n)$, alt problemlerin sonuçlarını birleştirmenin maliyetidir; b problemin kaç parçaya bölündüğünü, a bunlardan kaçıyla ilgilenildiğini gösterir. Ana teoremde a, b ve d arasındaki ilişkiye göre farklı karmaşıklık sonuçları çıkar. $n^{\log_b a}$ sonucu $a > b^d$ durumunda elde edilir. Dolayısıyla öğrenciden bu eşitsizliği sağlayan somut bir a, b, d seçip bağıntıyı yazması beklendi; bu yüzden cevap herkes için farklı olabilir.

Hocanın seçtiği bağıntı $T(n) = 3T(n/3) + 1$'dir: $a = 3$, $b = 3$, $d = 0$ ve $3 > 3^0 = 1$. Ağaçta her fonksiyon çağrısı, özyinelemeli çağrılar dışında 1 işlem yapar:

- Seviye 0: bir düğüm (n), 1 işlem.
- Seviye 1: üç düğüm ($n/3$), 3 işlem.
- Seviye 2: dokuz düğüm ($n/9$), 3^2 işlem.
- Ağaç $\log_3 n$ seviyede biter. Toplam: $\sum_{i=0}^{\log_3 n} 3^i$.

Hoca toplamın açılmasını beklemediğini, bu biçimde yazılmasının yeterli olduğunu söyledi. $T(n) = 2T(n/2)+1$ gibi işlem kolaylığı sağlayan seçimler de kabul edildi. Ancak ağacı somut sayılar yerine a, b harfleriyle çizen cevaplar, görüntüsü benzese de kabul edilmedi. Sorunun amacı ana teoremi tersten uygulamaktı.

1.2 Tümevarımla $O(n)$ sınaması

İkinci soru $T(n) = 2T(n-1) + 2$ bağıntısı için $T(n) = O(n)$ olup olmadığını tümevarımla (induction) araştırmayı istiyordu. Burada bir varsayım yapılır ve bu varsayımın doğru olup olmadığı kanıtlanmaya çalışılır. Bir küçük değer için $T(n-1) \leq c(n-1)$ olduğu varsayılır ve bağıntıda yerine konur (ekrandan, 00:10:00):

$$T(n) \leq 2c(n-1) + 2 = 2cn - 2c + 2 = cn + cn - 2c + 2$$

Bazı öğrenciler bu adımda hata yapıp $2cn - 2cn$ yazmıştı. Hedef $T(n) \leq cn$ biçimine ulaşmaktır. İfade şöyle yazılır:

$$T(n) \leq cn - (-cn + 2c - 2)$$

Eşitsizliğin tutması için parantez içindeki terimin negatif olmaması gerekir: $-cn + 2c - 2 \geq 0$. c'nin pozitif olması zorunlu olduğundan bu yalnız $n < 2$ için sağlanır. Büyük n ile ilgilenildiği için bağıntı $O(n)$'de çözülemez: $T(n) \notin O(n)$. Hoca bu açıklamanın yeterli olacağını söyledi.

> **Ek bilgi (derste anlatılmadı):** Tek bir tümevarım denemesinin başarısız olması tek başına "$O(n)$ değildir" demek için yeterli bir kanıt sayılmaz; ama bu bağıntıda sonuç gerçekten doğrudur. $T(0) = 0$ alınırsa bağıntı açılınca $T(n) = 2^{n+1} - 2$ bulunur (python ile ilk değerler 0, 2, 6, 14, 30, 62). Yani $T(n) = \Theta(2^n)$'dir ve $O(n)$ olamaz.

1.3 Amortize maliyet

Üçüncü soruda n işlemden oluşan bir dizi vardı: i 2'nin tam kuvvetiyse i'inci işlemin maliyeti i, değilse 1'dir. Hoca bu sorunun, derste gösterilen amortize analiz yöntemlerinden (toplam, muhasebe, potansiyel) birini gerektirmediğini, yalnız amortize maliyetin tanımını düşünmenin yeterli olduğunu vurguladı. Amortize maliyet hesaplanırken tek bir işlem yerine bir dizi işlemin toplam maliyeti bulunur ve işlem sayısına bölünür; örneğin pop ve push işlemlerinden oluşan bir yığın dizisinde olduğu gibi. Burada her işlemin maliyeti zaten verilmiştir; yapılması gereken bunları bir formülde birleştirmektir. Hoca bunu, amortize analizle ilgili bir örnek çözerken alt problem olarak çözdüğünü hatırlattı.

Tahtadaki çözüm (ekrandan, 00:13:10):

$$T(n) = \sum_{\lg i \in \mathbb{Z}} i + \sum_{\lg i \notin \mathbb{Z}} 1 \leq \sum_{i=1}^{\lg n} 2^i + \sum_{i=1}^{n} 1$$

Geometrik seri formülü $\sum_{i=0}^{n} a^i = \frac{a^{n+1}-1}{a-1}$ kullanılır (hoca bu formülü dönem başındaki slaytlarda verdiğini söyledi):

$$= \frac{2^{\lg n+1} - 1}{2-1} + n = 2n + n - 1 \approx 3n$$

Hoca, istenenin sonucun $3n$ ya da $5n$ olması değil, n'nin mertebesi cinsinden doğru ifade edilmesi olduğunu söyledi; katsayılar zaten göz ardı edilir.

> **Ders sonrası not:** Tahtada $3n$'nin yanına $\Theta(n)$ yazıldı. Bu, n işlemin toplam maliyetidir. İşlem başına amortize maliyet toplamın n'ye bölümüdür: $3n/n = 3$, yani $O(1)$. Ayrıca toplam $i = 1$'den başladığı için tam değer $2^{\lg n+1} - 2$'dir; sınır değişmez. Python ile sayıldığında $n = 8$ için toplam 19, $n = 1024$ için 3060 çıkar; ikisi de $3n$'nin altındadır.

1.4 Hiç çakışma olmama olasılığı

Dördüncü soru, n anahtarın tutulduğu, yuva sayısı $m = 2n$ olan ve düzgün dağılım varsayılan bir hash tablosunda hiç çakışma (collision) olmama olasılığını soruyordu (ekrandan, 00:16:50). Hoca bunun da derste bir örneğin içinde çözüldüğünü hatırlattı.

- Çakışmayı ikililer oluşturur. n anahtardan oluşan farklı çiftlerin sayısı $\binom{n}{2} = n(n-1)/2$'dir.
- Tabloda m yuva varken bir çiftin çakışma oluşturma olasılığı $1/m$'dir.
- Bir çiftin çakışma oluşturmama olasılığı $1 - 1/m$'dir.
- Bütün çiftlerin çakışmaması gerekir; olasılıklar çarpılır:

$$P(\text{hiç çakışma yok}) = \left(1 - \frac{1}{m}\right)^{n(n-1)/2}$$

Soruda $m = 2n$ verilmiştir; sonucu m ya da n cinsinden yazmak serbesttir, değer yerine konur.

> **Ek bilgi (derste anlatılmadı):** Derste kullanılan çarpım, çiftlerin çakışma olaylarını bağımsızmış gibi alan bir yaklaşımdır. Anahtarları sırayla yerleştiren tam hesap $\prod_{i=0}^{n-1} (1 - \frac{i}{m})$'dir. İki ifade de $1 - x \approx e^{-x}$ ile yaklaşık $e^{-n(n-1)/(2m)}$'e iner; $m = 2n$ için bu yaklaşık $e^{-(n-1)/4}$ olur.

Son soru Cuckoo hashing ile ilgiliydi. Bazı öğrencilerin örneğinde ekleme döngüye girmişti. Hoca, döngü oluştuğunun yazılmasının yeterli olduğunu söyledi ve bu soruyu çözmedi.

2. İkili Arama Ağaçlarına Giriş

Dersin yeni konusu dengeli ağaçlardır. Hoca, arka plan oluşsun diye önce ikili arama ağacını (binary search tree) hatırlattı. Lisans derslerinde genellikle ekleme ve arama görülür; bu derste silme de ayrıntılı işlendi.

İkili arama ağacı arama, minimum bulma, maksimum bulma, öncül (predecessor), ardıl (successor), ekleme ve silme gibi işlemleri destekler. Bu işlemlerin karmaşıklığı ağacın yüksekliği h ile orantılıdır. Her düğümün en fazla iki çocuğu olduğu için rastgele oluşturulmuş bir ağaçta yükseklik $\Theta(\lg n)$ beklenir. Ama elemanlar sıralı gelirse ağaç bir zincir, yani bağlı liste hâline gelir ve karmaşıklık $\Theta(n)$'e çıkar (ekrandan, 00:26:25). Bunu ortadan kaldırmak için kendini değiştirerek hep dengeli kalan ağaç türleri vardır; dersin hedefi bunlardır.

Tahtadaki küçük örnek ağaçta kök 6'dır; solunda 5 (çocukları 2 ve 5), sağında 7 (sağ çocuğu 8) bulunur. Bu ağaçta 5 iki kez geçer; örnekler genelde anahtarlar farklı (distinct) varsayılarak çözülür, tekrar eden anahtarlar Bölüm 7'de ele alındı. Temel özellik şudur: x bir düğüm olmak üzere,

- y, x'in sol alt ağacındaysa `y.key <= x.key`,
- y, x'in sağ alt ağacındaysa `y.key >= x.key`.

Yani sağdaki düğümler ebeveynden büyük, soldakiler küçüktür. Gerçeklemede, örneğin C ile, her düğüm `parent`, `left` ve `right` işaretçilerini tutar. Bağ çift yönlüdür: 7'nin `right`'ı 8 ise 8'in `parent`'ı 7'dir. Arama kökten aşağı iner, ama silme gibi işlemlerde yapraktan köke doğru gitmek ve bir düğümün ebeveyninin sol mu sağ çocuğu olduğunu kontrol etmek gerekir; çift yönlü bağ bu yüzden kullanışlıdır.

Hoca, şekillerde düğüm içine yazılan sayının düğümün kendisi değil anahtarı (key) olduğunu, anlaşılması kolay olsun diye böyle gösterdiğini belirtti.

3. Arama, Minimum ve Maksimum

Arama özyinelemeli yazıldı (ekrandan, 00:26:25):

```
TREE-SEARCH(x, k)
  if x == NIL or k == x.key
      return x
  if k < x.key
      return TREE-SEARCH(x.left, k)
  else return TREE-SEARCH(x.right, k)
```

Kök verilir; NIL işaretçisine gelinirse ya da bulunulan düğümün anahtarı aranan anahtarsa o düğüm döndürülür. Değilse dallanma yönü belirlenir: aranan anahtar küçükse sol çocukla, değilse sağ çocukla devam edilir. Aynı algoritma döngüyle (iteratif) de yazılabilir. Karmaşıklık ağacın derinliği kadardır: $O(h)$.

Minimum ve maksimum için yalnız biri yazıldı (ekrandan, 00:30:00):

```
TREE-MINIMUM(x)
  while x.left != NIL
      x = x.left
  return x
```

x bir alt ağacın kökü de olabilir. Küçükler solda olduğu için NIL ile karşılaşılmadığı sürece sola gidilir. Maksimum için `x.left` yerine `x.right` kontrol edilir ve `x = x.right` yapılır. İki işlemde de ağacın yüksekliği boyunca ilerlenebileceği için karmaşıklık $O(h)$'dir.

4. Ardıl ve Öncül

Ardıl ve öncül, ağaçtan silme yaparken önemli olan iki kavramdır. Anahtarlar farklıyken bir x düğümünün ardılı, x'ten büyük en küçük düğümdür. Tahtadaki ağaçta (kök 20; solda 16, onun altında 15 ve 18; 15'in altında 14; 18'in altında 17 ve 19; sağda 30 ve onun altında 27) 16'nın ardılı 17'dir (ekrandan, 00:30:00). Bu bilgi şöyle kullanılır: 16 silinmek istendiğinde 17, 16'nın yerine taşınır ve silme tamamlanır. Silmede başka durumlar da vardır; bunlar Bölüm 6'da.

Ardıl kodu iki parçadan oluşur (ekrandan, 00:34:35):

```
TREE-SUCCESSOR(x)
  if x.right != NIL
      return TREE-MINIMUM(x.right)
  y = x.p
  while y != NIL and x == y.right
      x = y
      y = y.p
  return y
```

1. Birinci parça x'in sağ kolunun boş olup olmadığına bakar. Boş değilse sağ alt ağacın minimumu aranan değerdir; az önce yazılan `TREE-MINIMUM` fonksiyonu x'in sağ çocuğuyla çağrılır.
2. İkinci parça sağ kol boşken çalışır: ebeveyne doğru yukarı çıkılır. Her yukarı adımda sağa mı sola mı dönüldüğüne bakılır. Sola doğru gidilen (yani sağ çocuk olarak yukarı çıkılan) her adımda kendinden küçük bir düğümle karşılaşılır. İlk kez sağa dönüldüğünde gelinen düğüm, bulunulan noktadan büyüktür; ardıl odur.

Örnek ağaç: kök 15; sol çocuk 6 (çocukları 3 ve 7), 3'ün çocukları 2 ve 4, 7'nin sağ çocuğu 13, 13'ün sol çocuğu 9; sağ çocuk 18 (çocukları 17 ve 20).

- $x = 6$: sağ alt ağaç boş değil, ardıl bu alt ağacın en küçüğü olan 7'dir.
- $x = 13$: sağ alt ağaç boştur. $y = 7$ (13'ün ebeveyni) ile başlanır.
  - $y = 7 \neq$ NIL ve $13 = 7.right$: $x = 7$, $y = 6$.
  - $y = 6 \neq$ NIL ve $7 = 6.right$: $x = 6$, $y = 15$.

- $6 \neq 15.right$ (6, 15'in sol çocuğu): döngü biter, ardıl 15'tir.

Hoca bu işlemin karmaşıklığını sınıfa sordu. Hangi durum çalışırsa çalışsın fark etmez: birinci durumda en kötü ihtimalle ağacın yüksekliği boyunca aşağı inilir, ikinci durumda bir yapraktan köke kadar çıkmak gerekebilir. İlgilenilen şey en fazla adım sayısıdır ve ikisinde de bu ağaç yüksekliğidir: $O(h)$. Ağaç dengeliyse bütün bu işlemler $O(\lg n)$'de yapılır.

Öncül, bir düğümden küçük en büyük elemandır; yani düğümün sol tarafındaki en büyük eleman. Yukarıdaki ağaçta 6'nın öncülü 4'tür. Öncül de silmede kullanılır; silme için iki kavramı da kullanan farklı stratejiler vardır. Hoca, öncülü kullanacak bir silmenin genelde özyinelemeli bir yaklaşım gerektirebileceğini söyledi.

5. Ekleme

Ağaca bir z düğümünün eklenmesi şöyledir (ekrandan, 00:41:10):

```
TREE-INSERT(T, z)
  y = NIL
  x = T.root
  while x != NIL
      y = x
      if z.key < x.key
          x = x.left
      else x = x.right
  z.p = y
  if y == NIL
      T.root = z
  elseif z.key < y.key
      y.left = z
  else y.right = z
```

x, ağaç üzerinde her seferinde sola ya da sağa ilerlemek için kullanılan ara değişkendir. Kökten başlanır ve aramadaki gibi ilerlenir; ama eklemede ağacın sonuna kadar inilir ve bulunan yere yerleştirilir. Yapraklara ulaşılmadığı, yani NIL görülmediği sürece eklenecek anahtar bulunulan düğümün anahtarıyla karşılaştırılır; küçükse `x = x.left`, değilse `x = x.right` yapılır.

Döngüden çıkma sebebi x'in NIL ile karşılaşmasıdır. Bu durumda y, NIL'den bir önceki düğümü gösterir; dolayısıyla z'nin ebeveyni y olmalıdır. y NIL'e eşitse ağaç boştur ve kök z yapılır. Ağaç boş değilse z'nin y'nin soluna mı sağına mı yerleşeceği bilinmez; anahtarlar karşılaştırılır: z'ninki küçükse sola, değilse sağa. Karmaşıklık, arama gibi kökten yapraklara indiği için ağacın yüksekliğiyle orantılıdır: $O(h)$.

6. Silme ve TRANSPLANT

6.1 Üç durum

Silmede üç farklı durum vardır:

1. Silinecek z'nin hiç çocuğu yoktur. Örneğin q düğümünün çocuğu z olsun; z basitçe kaldırılır. q ile z arasındaki bağ silinir ve q'nun ilgili çocuk işaretçisi NIL'i gösterecek şekilde güncellenir.
2. z'nin tek çocuğu vardır. z'nin çocuğu q'nun çocuğu olacak şekilde bağ kurulur: hem q'nun çocuk işaretçisi z'nin çocuğunu gösterir, hem de çocuğun ebeveyn işaretçisi q'ya çevrilir.
3. z'nin iki çocuğu vardır. Bu durum biraz daha incedir. z'nin ardılı y bulunur ve y, z'nin yerine geçirilir ("have y take z's position"); z'nin sağ alt ağacı y'nin yeni sağ alt ağacı olur.

6.2 TRANSPLANT

Bütün durumlarda, özellikle üçüncüsünde, kullanılacak bir yardımcı fonksiyon yazıldı. `TRANSPLANT(T, u, v)`, u köküne sahip alt ağacı v köküne sahip alt ağaçla değiştirir; değişimden sonra u'nun ebeveyni v'nin ebeveyni olur (ekrandan, 00:47:10):

```
TRANSPLANT(T, u, v)
1  if u.p == NIL
2      T.root = v
3  elseif u == u.p.left
4      u.p.left = v
5  else u.p.right = v
6  if v != NIL
7      v.p = u.p
```

- İlk iki satır u'nun kök olduğu durumu karşılar: u kökse artık v kök olur.
- u kök değilse ebeveyninin sol çocuğu mu diye bakılır; öyleyse ebeveynin `left` işaretçisi v'yi gösterir. Değilse u sağ çocuktur ve `right` işaretçisi v'ye çevrilir.
- 6. ve 7. satırlar çift yönlü bağı kurar: v NIL değilse ebeveyni u'nun ebeveyni olur.

`TRANSPLANT`, v'nin sol ya da sağındaki elemanlarla ilgilenmez; yalnız u'nun yerine v'yi koyar. Çocukların düzenlenmesi silme işleminin içinde yapılır.

6.3 TREE-DELETE ve dört örnek durum

Silme kodu ve hocanın dört örnek durumu (ekrandan, 00:50:00 ve 01:00:00):

```
TREE-DELETE(T, z)
1   if z.left == NIL
2       TRANSPLANT(T, z, z.right)
3   elseif z.right == NIL
4       TRANSPLANT(T, z, z.left)
5   else y = TREE-MINIMUM(z.right)
6       if y.p != z
7           TRANSPLANT(T, y, y.right)
8           y.right = z.right
9           y.right.p = y
10      TRANSPLANT(T, z, y)
11      y.left = z.left
12      y.left.p = y
```

Üç durum dört örnekle görselleştirildi. Şekillerde q, z'nin ebeveynidir; l ve r, z'nin sol ve sağ çocuklarıdır.

- **S1:** z'nin sol çocuğu NIL, sağ çocuğu r. Satır 1-2 çalışır: `TRANSPLANT`'a ikinci parametre olarak r gönderilir, r z'nin yerine geçer ve bağlar kurulur. (Hiç çocuk yoksa r de NIL'dir ve aynı satırlar z'yi kaldırır.)
- **S2:** z'nin sağ çocuğu NIL, sol çocuğu l. Satır 3-4 çalışır ve l, z'nin yerine geçer.
- **S3:** z'nin iki çocuğu vardır ve ardıl y, z'nin sağ çocuğunun kendisidir; y'nin sol kolu boştur, sağında x vardır. Satır 5'te `TREE-MINIMUM(z.right)` ile ardıl bulunur ve y'ye atanır. y'nin ebeveyni z olduğu için satır 6'daki koşula girilmez; doğrudan satır 10'daki `TRANSPLANT(T, z, y)` çalışır ve y, z'nin yerine yazılır. Satır 11-12 l'yi y'nin sol çocuğu olarak bağlar. y'nin sağ tarafı (x) olduğu gibi kalır. Sonuç: q'nun altında y, onun solunda l, sağında x.
- **S4:** z'nin iki çocuğu vardır ama ardıl hemen sağındaki düğüm değildir; r ile başlayan sağ alt ağacın içinde bir yerdedir (şekilde kesik çizgilerle gösterildi; r ile y arasında başka düğümler olabilir). y'nin sol kolu yine boştur.

Hoca burada sınıfa sordu: kod bu noktaya geldiyse y'nin sol kolu neden boş olmak zorundadır? Cevap ardıl tanımından gelir. Ardıl iki durumda bulunuyordu: sağ kol NIL ise ağaçta yukarı çıkılıyor, değilse sağ alt ağacın minimumu bulunuyordu. Silmede z'nin sağ kolunun boş olduğu durum satır 1-4'te zaten çözülmüştür; kod satır 5'e geldiyse z'nin sağ kolu NIL değildir ve ardıl sağ alt ağacın minimumudur. Minimumun sol çocuğu olamaz; dolayısıyla y'nin sol kolu boştur.

S4'te adımlar şöyledir:

1. Satır 5: ardıl bulunur.
2. Satır 6: ardıl z'nin sağ çocuğu mu diye bakılır; S3'te öyleydi, burada değil. Koşula girilir.
3. Satır 7: `TRANSPLANT(T, y, y.right)`; y'nin sağ çocuğu x, y'nin yerine geçer.
4. Satır 8-9: y'nin sağına z'nin sağ alt ağacı (r) bağlanır ve r'nin ebeveyni y yapılır.
5. Satır 10: ikinci `TRANSPLANT` ile y, z'nin yerine geçer.

6. Satır 11-12: z'nin sol çocuğu l, y'nin sol çocuğu olarak bağlanır.

Sonuçta ağaç q, y, r, x sırasıyla bağlanmış olur ve l, y'nin sol kolundadır; z aradan çıkarılmıştır.

Karmaşıklık: `TRANSPLANT` dışında her şey sabit zamanda yapılır ve `TRANSPLANT` da sabit zamanlıdır. Yalnız `TREE-MINIMUM` ağaçta aşağı inmeyi gerektirir. Hoca önce "$O(n)$" dedi, hemen düzeltip silme karmaşıklığının $O(h)$ olduğunu söyledi.

7. Tekrar Eden Anahtarlar

Buraya kadarki algoritmaların hepsi anahtarların farklı olduğu durum için geçerlidir. Hoca, aynı anahtarların bulunabildiği bir ağaç gerçeklemek istendiğinde ne yapılabileceğini sordu. Örnek dizi: 10, 8, 15, 8, 7, 8, 8 (ekrandan, 01:01:40).

- **Birinci seçenek:** Bir kural belirlenir: aynı elemanla ikinci kez karşılaşılınca hep sağa (ya da hep sola) gidilir. 10, 15 ve 8 yerleştirildikten sonra ikinci 8 gelince ilk 8 bulunur ve sağına konur; sonraki 8'ler de hep sağa gider.
- **İkinci seçenek:** Her düğümde o anahtardan kaç tane olduğu tutulur. Örneğin bir tane 10, dört tane 8 gibi.
- **Üçüncü seçenek:** Düğüme bir bayrak (flag) eklenir. Düğümden her geçildiğinde, aynı eleman yerleştirilecekse bayrağa göre bir kez sağa, bir kez sola konur.

Seçeneklerin birbirine göre avantajları vardır. Bayraklı yöntem ağacın daha dengeli kalmasını sağlar ve arama kolaylaşır. Sayaç tutan yöntem derinliği düşük bir ağaç çıkarır, ama düğümlerde ek bilgi tutulur. Hangisinin seçileceği uygulamaya bağlıdır.

> **Ders sonrası not:** Tahtada 7, ikinci 8'in sol çocuğu olarak çizilmiş görünüyor (ekrandan, 01:01:40). "Eşitler sağa" kuralıyla 7 eklenirken 10'dan sola, ilk 8'den de $7 < 8$ olduğu için sola gidilir; yani 7, ilk 8'in sol çocuğu olur. Sağdaki 8 zinciri bundan etkilenmez.

8. Radix Ağaçları

Aradan sonra hoca radix ağaçlarını (radix tree) anlattı. Radix ağacı, ikili dizgileri (bit strings) ya da gerçek karakter dizgilerini tutmak için oluşturulmuş bir ağaç türüdür ve sözlük (dictionary) gerçeklemeleri için oldukça kullanışlıdır.

Tahtadaki örnek küme: 1011, 10, 011, 100, 0 (ekrandan, 01:22:10). İkili arama ağaçlarına benzer biçimde dallanma bitin değerine göre yapılır: 0'lar sola, 1'ler sağa gider. Karakter dizgilerinde ise karakterlerin sözlükteki (ASCII) sırası düşünülür.

Ağaç gri ve beyaz düğümlerden oluşur. Gri düğümler NIL işaretçisidir; bir şey ifade etmezler, yalnız yolun parçasıdırlar. Beyaz düğümler kümedeki bir dizgiyi tutar; tasarıma bağlı olarak örneğin dizginin sözlükteki anlamına bir işaretçi tutabilir. Örnek ağaçta:

- Kök gridir.
- Kökten 0 ile inilen düğüm beyazdır ve "0"ı tutar.
- Ondan 1 ile inilen düğüm (01) gridir; ondan 1 ile inilen düğüm beyazdır ve "011"i tutar.
- Kökten 1 ile inilen düğüm (1) gridir; ondan 0 ile inilen düğüm beyazdır ve "10"u tutar.
- "10"dan 0 ile inilen düğüm "100"ü tutar. "10"dan 1 ile inilen düğüm (101) gridir; kümede 101 yoktur ama 1011 vardır. Ondan 1 ile inilen düğüm "1011"i tutar.

Radix ağacının ikili arama ağacından farkı, her küme için tek (unique) bir yapı ortaya çıkmasıdır; dizgiler hangi sırayla eklenirse eklensin aynı ağaç oluşur. İkili arama ağacında ise farklı ağaçlar aynı kümeyi tutabilir. Ayrıca yer tasarrufu sağlar.

İngilizce kelimelerden oluşan küçük bir sözlük için de böyle bir ağaç kurulabilir. Bu durumda düğümlerin iki değil daha fazla çocuğu olur; bu sayıya dallanma faktörü (branching factor) denir. Bir düğümün en fazla çocuk sayısı alfabedeki karakter sayısı kadardır.

8.1 Arama

Arama üç farklı biçimde sonlanabilir. Aranan dizginin her karakteri izlenerek ağaçta ilerlenir:

- Beyaz bir düğümde biter: dizgi bulunmuştur. Örneğin 011 için sola, sağa, sağa gidilir ve beyaz düğüme varılır; artık anlamı ekrana yazdırılabilir.
- Gri bir düğümde biter: ağaç bu dizgiyi modelleyebilir ama sözlükte yoktur. Örneğin 01.
- NIL'e çarpar: dizgi ağaçta modellenmemiştir bile. Örneğin 0 ile sola gidildikten sonra 0 ile yeniden sola gidilmek istendiğinde karşıya NIL çıkar.

Hoca gri düğümü bir sözlük benzetmesiyle açıkladı: sözlükte "araba" varken "ara" yoksa, "ara" için ağaçta yol vardır ama o düğüm gridir.

Soru: Gri düğümler hep arada mı olmak zorundadır? **Cevap:** Hayır. Ağaç oluşturulurken her düğüm için ya NIL işaretçisi tutulur ya da düğüm gerçekten bir şeye (örneğin sözlükteki anlama) işaret eder.

8.2 Ekleme

Eklemede de iki durum vardır (ekrandan, 01:28:50):

- Dizgi ağaçta zaten modellenmiştir ama düğümü gridir. Örneğin 101 eklenirken 101 aranır ve gri düğüm bulunur; düğüm beyaza çevrilir. Artık sözlükte bu sözcük vardır.
- Dizgi için hiç düğüm yoktur. Örneğin 11 eklenirken yeni bir düğüm oluşturulur ve 1 düğümünün sağ çocuğu olarak yerleştirilir.

8.3 Silme

Silmede de iki durum vardır:

- Silinecek dizgi bir ara düğümdeyse düğüm silinmez, griye çevrilir; yani işaretçisi NIL'i gösterir. Çünkü o düğüm alttaki yapraklara giden yolun parçasıdır. Örneğin 0 silinirken 0 düğümü gri yapılır.
- Silinecek dizgi bir yapraktaysa yaprak silinir ve özyinelemeli olarak yukarı doğru çıkılıp onunla bağlantılı, artık bir şey tutmayan gri düğümler de kaldırılır. Örneğin 1011 silinince 1011 yaprağı ve onun üstündeki gri 101 düğümü kaldırılır.

9. Kırmızı-Siyah Ağaçlar

İkili arama ağacında arama, ekleme, ardıl gibi bütün işlemler ağaç derinliği h cinsinden $O(h)$'dir. Ağaç dengeli değilse bağlı listeye döner ve derinlik eleman sayısına çıkar. Dengeli bir ağacı garanti eden yaklaşımlardan biri kırmızı-siyah ağaçtır (red-black tree). Hoca bu yapının lisansta görüldüğünü sordu ve onayladı; kırmızı-siyah ağaçları B-ağaçlarının biraz daha basit bir hâli olarak andı. Ağaç her zaman dengeli olduğu için bütün işlemler $O(\lg n)$'de yapılır.

Ağaç adını düğümlerin renginden alır: her düğüm ya kırmızı ya siyahtır. Düğümlere renk verilmesi, ağacın dengeli olmasını sağlayan kuralların denetlenmesine yarar. Ekleme ya da silme yapıldığında kurallar bozulabilir; bu yüzden her değişiklikten sonra ağaç düzeltilir ve kuralların yeniden sağlandığından emin olunur.

Kitap şeklinde siyahlar koyu, kırmızılar gri gösterilir (ekrandan, 01:30:00). Yine sağda büyükler, solda küçükler bulunur; ikili bir ağaçtır. Her düğümden sonra NIL'i gösteren yapraklar vardır; bunlar işlemlerin gerçeklenmesi için ağaca konur.

Kırmızı-siyah ağacın uyması gereken kurallar:

1. Her düğüm ya kırmızı ya siyahtır.
2. Kök siyahtır.
3. Her yaprak (NIL) siyahtır.
4. Bir düğüm kırmızıysa iki çocuğu da siyahtır.
5. Her düğüm için, o düğümden yapraklara inen bütün basit yollarda siyah düğüm sayısı aynıdır.

Beşinci kural ağacın dengeli olmasını sağlar. Şekilde düğümlerin yanına küçük sayılar yazılmıştır; bunlar siyah yükseklik (black height) değerleridir: düğümden aşağı, bir yaprağa kadar olan yoldaki siyah düğümlerin sayısı. Düğümün kendisi sayılmaz. Şekilde 3 ve 7 için 1, 10 ve

14 için 2, 17 ve 26 için 3'tür. Bir düğümden aşağı inen farklı yolların hepsinde siyah yükseklik aynıdır; denge budur.

> **Hocanın vurgusu:** Bu özelliklerin her değişiklikten sonra korunup korunmadığı denetlenir; ağaç böylece her seferinde dengeli hâlde tutulur.

10. Kırmızı-Siyah Ağacın Yüksekliği

Kitaptaki lemma: n iç düğümlü bir kırmızı-siyah ağacın yüksekliği en fazla $2\lg(n+1)$'dir (ekrandan, 01:40:00). Hoca kanıtı sınıfa önerdi, sonra birlikte yaptı.

$bh(x)$, x düğümünün siyah yüksekliğidir. İlk iddia: x köklü bir alt ağaçta, yapraklar hariç, en az $2^{bh(x)}-1$ iç düğüm vardır. Hoca bunu küçük bir örnekle sınadı: 10 (siyah), onun sol çocuğu 7 (siyah), onun sol çocuğu 3 (kırmızı) ve altında NIL.

- $bh(3)=1$: altında yalnız siyah NIL vardır.
- $bh(7)=1$: 3 kırmızıdır, yalnız NIL sayılır.
- $bh(10)=2$: yolda 7 ve NIL siyahtır.
- $x=10$ için $2^2-1=3$; alt ağaçta gerçekten 3 düğüm vardır (10, 7, 3).
- $x=3$ için $2^1-1=1$; alt ağaçta 1 düğüm vardır.

İkinci adım: kökten yaprağa giden herhangi bir yolun en az yarısı siyah olmak zorundadır. Bu kural 4'ün sonucudur: kırmızı bir düğümün çocukları siyah olmak zorunda olduğundan iki kırmızı art arda gelemez. Yarıdan daha fazlası da siyah olabilir. Böylece kökün siyah yüksekliği için $bh(\text{kök}) \geq h/2$ yazılır.

> **Ders sonrası not:** Hoca bu noktada "her bir red için iki tane kırmızı ortaya çıkıyor" dedi (01:38:36). Doğrusu kural 4'tür: her kırmızı düğümün iki çocuğu da siyahtır.

İki adım birleştirilir. Ağaçta en az $2^{h/2}-1$ düğüm olduğuna göre toplam düğüm sayısı bundan büyük olmak zorundadır (ekrandan, 01:44:55):

$$n \geq 2^{h/2}-1 \;\Rightarrow\; n+1 \geq 2^{h/2}$$

İki tarafın 2 tabanında logaritması alınır:

$$\lg(n+1) \geq h/2 \;\Rightarrow\; h \leq 2\lg(n+1)$$

Katsayı göz ardı edilince yükseklik $\Theta(\lg n)$ mertebesindedir. Böylece bu ağacın dengeli olma özelliği kanıtlanmış olur.

> **Ek bilgi (derste anlatılmadı):** Derste ilk iddia (x köklü alt ağaçta en az $2^{bh(x)}-1$ iç düğüm) yalnız örnekle gösterildi. Ders kitabında bu büyük olasılıkla x'in yüksekliği üzerinden tümevarımla kanıtlanır: her çocuğun siyah yüksekliği en az $bh(x)-1$'dir, iki çocuk alt ağacında en az $2\cdot(2^{bh(x)-1}-1)$ düğüm bulunur ve x'in kendisi de eklenince $2^{bh(x)}-1$ elde edilir.

11. Döndürme ve Kırmızı-Siyah Ağaca Ekleme

11.1 Döndürme

Ekleme ve silmeden sonra kırmızı-siyah ağaçta değişiklik yapmak gerekir. Bunun için döndürme (rotation) denen küçük işlem adımları kullanılır. Döndürme ağacın yapısını değiştirir; yapılan iş işaretçiler üzerinde oynamaktır. İki çeşidi vardır:

- **Sol döndürme (left rotation):** x üzerinde yapılır; x sola, yani aşağı iner. x'in sağ çocuğu y (NIL olmamalı) alt ağacın yeni kökü olur. y'nin sol alt ağacı x'in sağ alt ağacına geçer.
- **Sağ döndürme (right rotation):** y üzerinde yapılır ve y sağa geçer.

İkisi birbirinin tersidir (tümleyen işlemler). Çocuklar yapıyı bozmayacak şekilde yeni yerlerine yerleşir. Kitap örneğinde (ekrandan, 01:44:55) 11 (x) üzerinde sol döndürme yapılınca 18 (y) yukarı çıkar, 11 aşağı iner; 18'in sol alt ağacı olan 14 (çocukları 12 ve 17), 11'in sağ alt ağacı

olur. Ağacın diğer kısımları değişmez. Şeklin açıklamasına göre iki ağacın sıralı (inorder) dolaşımı aynı anahtar listesini verir.

Hoca döndürmenin kodunu yazmadığını, örnekler üzerinden gideceğini söyledi.

11.2 RB-INSERT

Kırmızı-siyah ağaca ekleme, normal ikili arama ağacı eklemesiyle (Bölüm 5) belli bir satıra kadar tamamen aynıdır. Asıl işi sondaki `RB-INSERT-FIXUP` fonksiyonu yapar: renk değiştirmeler ve döndürmelerle ağacı düzeltir. Ondan önceki üç satır yeni düğümün `left` ve `right` çocuklarını NIL yapar ve rengini kırmızı atar (ekrandan, 01:50:00):

```
z.left = T.nil
z.right = T.nil
z.color = RED
RB-INSERT-FIXUP(T, z)
```

Yeni düğüm her zaman kırmızı eklenir. Sonra bağlı olduğu ebeveyn ve komşu düğümlerin renkleri değişebilir. Hoca fixup kodunun ders kitabında ve internette bulunduğunu, kural tabanlı bir işlem olduğu için kodu anlatmayacağını, sonucu kurallar üzerinden şekilde göstereceğini söyledi.

Soru: Yeni düğüm neden kırmızı ekleniyor? **Cevap:** Algoritma gereği hep kırmızı eklenir. Siyah eklenseydi kural 5 en baştan bozulurdu; bu yüzden kırmızı eklemek daha mantıklıdır. Hoca, siyah ekleyen bir algoritma yazmak isteyenin kurallarını ona göre baştan tanımlaması gerektiğini söyledi.

11.3 Ekleme örneği

Kitap şeklinde 4 ekleniyor (ekrandan, 01:50:00). Ağaçta kök 11 (siyah); 11'in solunda 2 (kırmızı), sağında 14 (siyah) ve 14'ün sağında 15 (kırmızı); 2'nin solunda 1 (siyah), sağında 7 (siyah); 7'nin solunda 5 (kırmızı), sağında 8 (kırmızı) vardır.

1. 4, ikili arama ağacındaki gibi yerine götürülür ve 5'in sol çocuğu olarak kırmızı eklenir. Kurallara bakılır: 5'te sorun yoktur, ama 4 ve ebeveyni 5 ikisi de kırmızıdır; kural 4 bozulmuştur.
2. **Durum 1:** En basit çözüm 5'i siyah yapmaktır. Ama yalnız 5 siyah yapılırsa yollardaki siyah sayıları eşit olmaz ve kural 5 bozulur. Bu yüzden 5 ve 8 (4'ün amcası) siyaha, 7 kırmızıya boyanır. Artık incelenen düğüm 7'dir.
3. **Durum 2:** Şimdi 7 ve ebeveyni 2'nin ikisi de kırmızıdır; yine kural ihlali vardır. 2 üzerinde sol döndürme yapılır: 2 sola iner, 7 onun ebeveyni olur, 7'nin sol çocuğu 5, 2'nin sağ çocuğu olur.
4. **Durum 3:** Bu kez 7 ile 11 arasında sağ döndürme yapılır: 11, 7'nin sağına geçer. Kural 5 bozulmasın diye renkler de değiştirilir: 11 kırmızıya boyanır, 7 siyah olur. Sonuçta kök 7'dir ve bütün kurallar sağlanır.

   Ek bilgi (derste anlatılmadı): Şekildeki durum adları büyük olasılıkla kitaptaki fixup koduna göredir: Durum 1, z'nin amcası kırmızıyken yeniden boyamadır; Durum 2, amca siyah ve z sağ çocukken yapılan sol döndürmedir; Durum 3, amca siyah ve z sol çocukken yapılan yeniden boyama ve sağ döndürmedir. Durum 1 sorunu ağaçta iki seviye yukarı taşıyabilir; Durum 2 ve 3 en fazla iki döndürmeyle işi bitirir.

Karmaşıklık: fixup öncesi kısım normal eklemedeki gibi $O(\lg n)$, fixup da $O(\lg n)$'dir; toplam $O(\lg n)$. Ağaç dengeli bir yapıya getirilmiş olur.

12. B-Ağaçlarına Giriş

B-ağaçları (B-tree), kırmızı-siyah ağaçlar gibi dengeli arama ağaçlarıdır. Disk gibi ikincil depolama aygıtlarında doğrudan erişim için kullanılırlar. Kırmızı-siyah ağaçlara benzerler ama G/Ç (I/O) işlemlerini azaltmada daha iyidirler. Pek çok veritabanı sistemi bunların çeşitlerini kullanır.

Kırmızı-siyah ağaçlardan farkı, düğümlerin çok sayıda çocuğu olabilmesidir; hoca radix ağacında geçen dallanma faktörüne gönderme yaptı (önce yanlışlıkla "denge faktörü" deyip düzeltti). Böylece geniş ağaçlar oluşur.

Tahtadaki örnek, İngilizce sessiz harflerden oluşan bir B-ağacıdır (ekrandan, 01:56:40). Bir x iç düğümünde `x.n` anahtar varsa düğümün `x.n + 1` çocuğu olur:

- Kökte yalnız M vardır (1 anahtar); iki çocuğu vardır: M'den küçükler ve büyükler.
- Soldaki düğümde D ve H vardır (2 anahtar); üç çocuğu vardır: D'den küçükler (B C), D ile H arasındakiler (F G) ve H'den büyükler (J K L).
- Sağdaki düğümde Q, T ve X vardır (3 anahtar); dört çocuğu vardır: Q'dan küçükler (N P), Q ile T arasındakiler (R S), T ile X arasındakiler (V W) ve X'ten büyükler (Y Z).

Örneğin R aranırken köke gelinir; R, M'den büyük olduğu için sağa gidilir; Q T X düğümünde Q ile T arasındaki çocuğa inilir ve R bulunur.

Veriler çok büyük olduğunda bu yapı önemlidir, çünkü her disk erişimi zaman kaybettirir. B-ağaçları istenen sayfaları (page) diskten çeker, üzerlerinde değişiklik yapar ve bir seferde geri yazar. Algoritmanın çalışma zamanını diskten okuma ve diske yazma sayısı etkiler. Diskten bir seferde okunacak miktar, yani disk sayfası, düğümlerdeki anahtar sayısıyla doğru orantılıdır; düğüm genişliği donanımın mekanik kısıtlarından gelen sayfa boyutuna göre ayarlanır.

Hoca bir örnek verdi (ekrandan, 02:00:00): kökte 1000 anahtar olsun.

- Kök: 1 düğüm, 1000 anahtar; 1001 çocuk.
- Birinci seviye: 1001 düğüm, her birinde 1000 anahtar: $1\,001\,000$ anahtar. Her birinin yine 1001 çocuğu vardır.
- İkinci seviye: $1001 \times 1001 = 1\,002\,001$ düğüm, $1\,002\,001\,000$ anahtar.

Yüksekliği 2 ve dallanma faktörü 1001 olan bu ağaçta herhangi bir anahtara yalnız iki erişimle ulaşılır; bu yüzden yapı oldukça etkilidir. Bir öğrenci sayının kapasiteyle ilgisini sordu; hoca bunun ağacın toplam tutabileceği miktardan, yani kapasiteden söz ettiğini söyledi.

> **Ders sonrası not:** Hoca bu ağacın "bir milyondan fazla" anahtar tuttuğunu söyledi. Tahtadaki sayılara göre yalnız ikinci seviyede $1\,002\,001\,000$ anahtar vardır; üç seviyenin toplamı $1000 + 1\,001\,000 + 1\,002\,001\,000 = 1\,003\,003\,000$'dir (python ile doğrulandı). Yani doğru ifade "bir milyardan fazla"dır. Bu değer bütün düğümler tam doluyken geçerlidir.

Hoca B-ağaçlarının çeşitlerine ve ayrıntılarına sonraki derste devam edeceğini söyledi.

Örnek Sorular

Soru 1 (ekrandan, 00:06:40; sınav sorusu): Çalışma zamanı $n^{\log_b a}$ olan bir yineleme bağıntısı oluşturunuz. Bağıntının ağacını çiziniz; her seviyede gerçeklenecek işlem sayısını ve toplam işlem sayısını bulunuz.

> **Çözüm (derste verildi):** $T(n) = aT(n/b) + O(n^d)$ için $n^{\log_b a}$ sonucu $a > b^d$ durumunda çıkar. Örnek seçim: $T(n) = 3T(n/3) + 1$ ($a = b = 3$, $d = 0$). Seviye i'de 3^i düğüm vardır ve her biri 1 işlem yapar; seviye i'nin işlem sayısı 3^i'dir. Ağaç $\log_3 n$ seviyede biter. Toplam $\sum_{i=0}^{\log_3 n} 3^i$. (Python ile $n = 3^5$ için toplam $364 = (3n-1)/2$; yani $\Theta(n) = \Theta(n^{\log_3 3})$.)

Soru 2 (ekrandan, 00:10:00; sınav sorusu): $T(n) = 2T(n-1) + 2$ bağıntısı için $T(n) = O(n)$ olup olmadığını tümevarımla araştırınız.

> **Çözüm (derste verildi):** $T(n-1) \le c(n-1)$ varsayılır. $T(n) \le 2c(n-1) + 2 = cn - (-cn + 2c - 2)$. $T(n) \le cn$ için $-cn + 2c - 2 \ge 0$ gerekir; $c > 0$ iken bu yalnız $n < 2$ için sağlanır. Büyük n için tutmadığından $T(n) \notin O(n)$.

Soru 3 (ekrandan, 00:13:10; sınav sorusu): n işlemden oluşan bir dizide i 2'nin tam kuvvetiyse i'inci işlemin maliyeti i, aksi hâlde 1'dir. Amortize maliyeti bulunuz.

> **Çözüm (derste verildi):** $T(n) \le \sum_{i=1}^{\lg n} 2^i + \sum_{i=1}^{n} 1 \le \frac{2^{\lg n+1}-1}{2-1} + n = 3n - 1 \approx 3n$. Toplam $\Theta(n)$'dir; işlem başına amortize maliyet $3n/n$, yani $O(1)$'dir.

Soru 4 (ekrandan, 00:16:50; sınav sorusu): n adet anahtarın tutulduğu bir hash tablosunda yuva sayısı $m = 2n$'dir ve dağılım düzgündür. Hiç çakışma olmama olasılığı nedir?

> **Çözüm (derste verildi):** Farklı çift sayısı $\binom{n}{2} = n(n-1)/2$. Bir çiftin çakışma olasılığı $1/m$, çakışmama olasılığı $1-1/m$. Hiç çakışma olmama olasılığı $\left(1 - \frac{1}{m}\right)^{n(n-1)/2}$, burada $m = 2n$.

Soru 5 (derste sözlü, 00:55:39): `TREE-DELETE` satır 5'e geldiyse ardıl y'nin sol kolu neden boş olmak zorundadır?

> **Çözüm (derste verildi):** Satır 1-4, z'nin sağ kolunun NIL olduğu durumu zaten çözer. Satır 5'e gelindiyse z'nin sağ alt ağacı vardır ve ardıl bu alt ağacın minimumudur. Minimum düğümün sol çocuğu olamaz; olsaydı o çocuk daha küçük olurdu.

Soru 6 (derste sözlü, 00:34:45): `TREE-SUCCESSOR` fonksiyonunun karmaşıklığı nedir?

> **Çözüm (derste verildi):** Sağ alt ağaç doluysa minimuma kadar aşağı inilir, boşsa köke kadar yukarı çıkılabilir. İki durumda da adım sayısı en fazla ağacın yüksekliği kadardır: $O(h)$. Ağaç dengeliyse $O(\lg n)$.

Soru 7 (derste sözlü, 01:00:31): Anahtarların tekrar edebildiği bir ikili arama ağacı nasıl gerçeklenebilir?

> **Çözüm (derste verildi):** (1) Eşit anahtarlar hep sağa (ya da hep sola) konur. (2) Her düğümde o anahtarın sayısı tutulur. (3) Düğümde bir bayrak tutulur ve eşitler sırayla bir sağa bir sola konur. Sayaç ağacı sığ tutar ama ek bilgi ister; bayrak ağacı daha dengeli tutar.

Soru 8 (ekrandan, 01:40:00): n iç düğümlü bir kırmızı-siyah ağacın yüksekliğinin en fazla $2\lg(n+1)$ olduğunu gösteriniz.

> **Çözüm (derste verildi):** x köklü alt ağaçta en az $2^{bh(x)} - 1$ iç düğüm vardır. Kural 4'ten kökten yaprağa giden yolun en az yarısı siyahtır: $bh(\text{kök}) \geq h/2$. Böylece $n \geq 2^{h/2} - 1$, $\lg(n+1) \geq h/2$, yani $h \leq 2\lg(n+1)$.

Soru 9 (ekrandan, 02:00:00): Kökünde 1000 anahtar bulunan, her düğümü 1000 anahtarlı ve yüksekliği 2 olan bir B-ağacında kaç düğüm ve kaç anahtar vardır?

> **Çözüm (derste verildi, toplam ders sonrası kesinleştirildi):** Her düğümün 1001 çocuğu vardır. Seviye 0: 1 düğüm, 1000 anahtar. Seviye 1: 1001 düğüm, 1 001 000 anahtar. Seviye 2: 1 002 001 düğüm, 1 002 001 000 anahtar. Toplam 1 003 003 000 anahtar (python ile doğrulandı); hoca bu ağaçta herhangi bir anahtara iki erişimle ulaşıldığını söyledi.

Dersin Toparlaması

Ders iki kısımdan oluştu. İlk kısımda sınav soruları üzerinden ana teoremin tersten kullanımı, tümevarımla bir sınırın sınanması, amortize maliyetin tanımdan hesabı ve hash tablosunda çakışmama olasılığı tekrar edildi. İkinci kısımda dengeli ağaçlara hazırlık olarak ikili arama ağacının bütün işlemleri sözde koduyla yeniden kuruldu: arama, minimum, ardıl, ekleme ve `TRANSPLANT` ile silme. Bu işlemlerin hepsi $O(h)$'dir; sorun h'nin n'ye çıkabilmesidir.

Bu sorunu çözen yapılardan kırmızı-siyah ağaç beş kuralı, siyah yükseklik kavramı ve $h \leq 2\lg(n+1)$ kanıtıyla incelendi; döndürme ve ekleme sonrası düzeltme kitap örneği üzerinden gösterildi. Araya dizgi kümeleri için radix ağaçları girdi. Son olarak disk tabanlı sistemler için B-ağaçlarına giriş yapıldı: çok çocuklu düğümler, `x.n` anahtar ve `x.n + 1` çocuk ilişkisi, düğüm boyutunun disk sayfasına göre seçilmesi. Hoca B-ağaçlarının ayrıntılarına sonraki derste devam edeceğini söyledi.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.